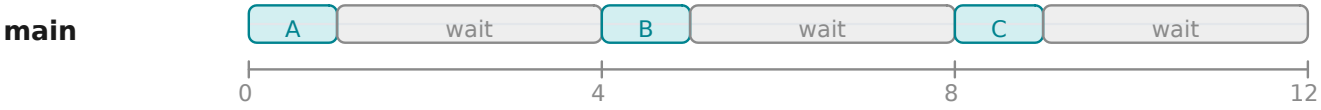
Chapter 16

Concurrency

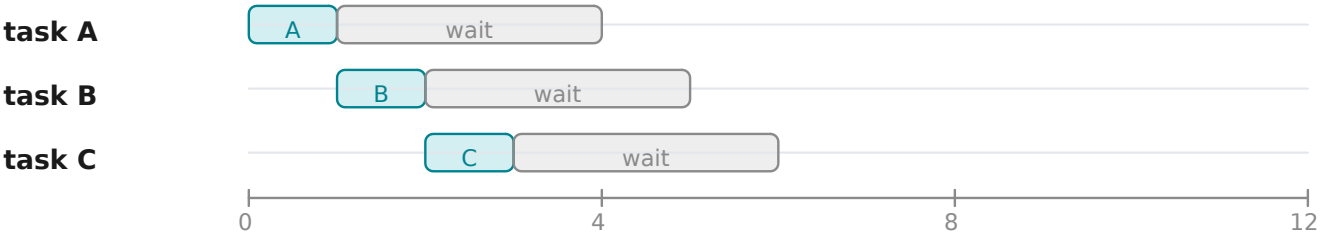
doing several things at once, and the bugs that come with it

Most programs spend their time **waiting**: for the network, the disk, a database. **Concurrency** means making progress on several tasks by working on one while the others wait. **Parallelism** means literally computing on several CPU cores at the same instant. Three jobs, each 1 unit of work plus 3 units of waiting for a server:

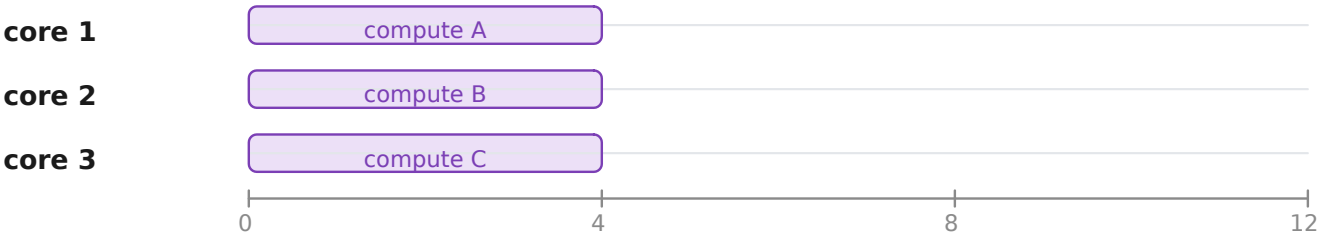
One at a time: 12 units



Concurrent (threads or asyncio, one core is enough): the waits overlap, 6 units



Parallel (processes, several cores): pure computing finishes 3× sooner



your program mostly...	use	why
waits on I/O (network, disk, DB)	threads, or `asyncio`	waiting releases the CPU for others
waits on thousands of connections	`asyncio`	a task costs KB; a thread costs MB
computes (CPU-bound pure Python)	processes (`ProcessPoolExecutor`)	each process has its own GIL
crunches arrays	NumPy (Ch 18)	its C loops already run outside the GIL

Threads: several flows in one program

A thread is a separate flow of execution sharing the program's memory. `start()` launches it, `join()` waits for it to finish. Here five slow "downloads" (a 0.2 s sleep stands in for the network) run one by one, then all at once:

```
1  import threading, time
2
3  def download(name, results):
4      time.sleep(0.2)                          # waiting: the thread gives up the CPU
5      results[name] = f"{name}: 200 OK"
6
7  names = ["dune", "emma", "ubik", "kim", "beloved"]
8
9  results, t0 = {}, time.perf_counter()
10 for n in names:
11     download(n, results)
12 print("one by one:", round(time.perf_counter() - t0, 1), "s")
13
14 results, t0 = {}, time.perf_counter()
15 threads = [threading.Thread(target=download, args=(n, results)) for n in names]
16 for t in threads:
17     t.start()
18 for t in threads:
19     t.join()                                 # wait for every thread
20 print("threaded:  ", round(time.perf_counter() - t0, 1), "s,", len(results), "results")
```

```
one by one: 1.0 s
threaded:   0.2 s, 5 results
```

line 4: I/O-bound: threads help

line 19: forget join() and you read results too early

Shared memory means races

Threads can be paused **between any two steps**. `balance = balance + 1` is three steps: read, add, write. If two threads interleave them, one update is lost:

step	thread 1	thread 2	balance
1	reads 5		5
2		reads 5	5
3	writes 5 + 1		6
4		writes 5 + 1	6 (should be 7)

```
1  balance = 0
2  lock = threading.Lock()
3
4  def deposit(times, use_lock):
5      global balance
6      for _ in range(times):
7          if use_lock:
8              with lock:                        # only one thread inside at a time
9                  balance = balance + 1
10         else:
11             current = balance
12             time.sleep(0)                     # any switch here loses updates
13             balance = current + 1
14
15 for use_lock in (False, True):
16     balance = 0
17     ts = [threading.Thread(target=deposit, args=(1000, use_lock)) for _ in range(4)]
18     for t in ts: t.start()
19     for t in ts: t.join()
20     print("with lock:" if use_lock else "no lock:  ", balance == 4000)
```

```
no lock:   False
with lock: True
```

line 8: the critical section

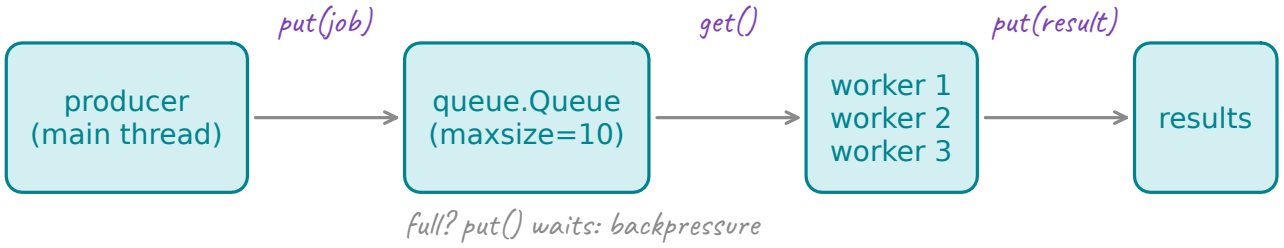
line 12: sleep(0) makes the race reliable to show

TRAP

A race may pass every test and fail once a week in production, because it depends on timing. The GIL does **not** save you: it stops two threads running bytecode at the same instant, but a thread can still be switched out between the read and the write. Protect every shared mutable value with a lock, or don't share it (next page).

Queues: share by passing messages

The safest shared data is none. Instead, threads hand work to each other through a `queue.Queue`, which does its own locking. Producers `put`, consumers `get`; a full or empty queue makes the caller wait:




```python
import queue, threading

jobs, results = queue.Queue(maxsize=10), queue.Queue()

def worker():
    while (n := jobs.get()) is not None:      # None = "no more work" (a sentinel)
        results.put((n, n * n))

workers = [threading.Thread(target=worker) for _ in range(3)]
for w in workers: w.start()
for n in range(8):
    jobs.put(n)
for _ in workers:
    jobs.put(None)                            # one sentinel per worker
for w in workers: w.join()
print(sorted(results.get() for _ in range(results.qsize())))
```

```
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25), (6, 36), (7, 49)]
```

line 6: get() blocks while empty

line 14: how workers know to stop

RULE OF THUMB

Prefer queues to locks. A lock protects data that many threads touch; a queue moves each piece of data so only one thread owns it at a time. Go's motto is the same idea: "don't communicate by sharing memory; share memory by communicating."

TRAP

Deadlock: thread 1 holds lock A and waits for B while thread 2 holds B and waits for A. Both wait forever. Avoid it by always taking locks in the same global order (practice 16.1), by holding only one at a time, or by using `lock.acquire(timeout=...)` and backing off.

Pools, and the GIL

Starting and joining threads by hand gets tedious. `concurrent.futures` gives you a pool of workers and a **future** for each job: a placeholder for a result that will exist later.

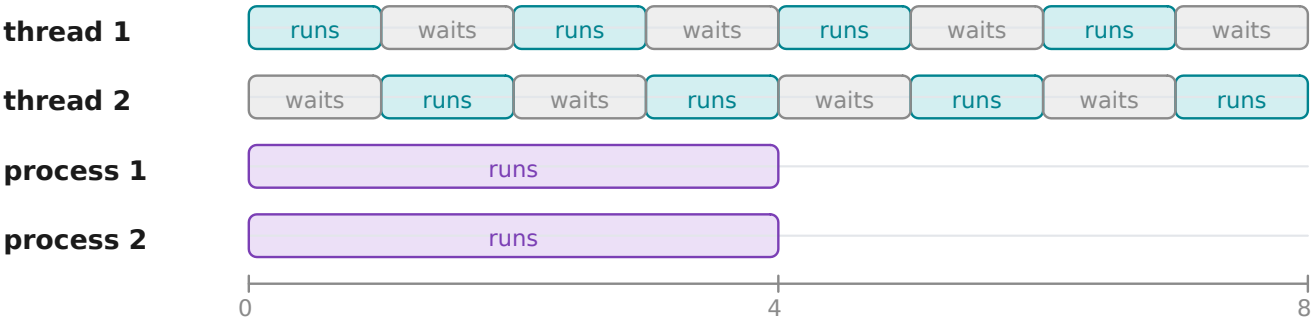
```
1  import time
2  from concurrent.futures import ThreadPoolExecutor, as_completed
3
4  def check(url):
5      time.sleep(0.1 if "slow" not in url else 0.3)
6      if "bad" in url:
7          raise ConnectionError(f"{url} refused")
8      return url, 200
9
10 urls = ["a.org", "slow.org", "bad.org", "b.org"]
11 with ThreadPoolExecutor(max_workers=4) as pool:
12     print(list(pool.map(check, ["a.org", "b.org"])))          # results in input order
13     futures = {pool.submit(check, u): u for u in urls}
14     for fut in as_completed(futures):                         # in finishing order
15         try:
16             print("done:", fut.result())
17         except ConnectionError as e:
18             print("failed:", e)
```

```
[('a.org', 200), ('b.org', 200)]
done: ('a.org', 200)
failed: bad.org refused
done: ('b.org', 200)
done: ('slow.org', 200)
```

line 13: submit returns a Future at once

line 16: result() re-raises the worker's error

So far all the waiting was I/O. For pure computation, CPython's **GIL** (global interpreter lock) lets only one thread run Python bytecode at a time, so threads don't speed it up; separate processes do:



Two CPU-bound threads take turns holding the GIL, so together they're no faster than one. Two processes each have their own interpreter and GIL, and finish the same work in half the time. This script was run while building this page:

primes.py

```python
import os, sys, time
from concurrent.futures import ProcessPoolExecutor, ThreadPoolExecutor

def count_primes(lo, hi):                       # pure CPU work: no waiting at all
    n = 0
    for k in range(max(lo, 2), hi):
        if all(k % d for d in range(2, int(k ** 0.5) + 1)):
            n += 1
    return n

CHUNKS = [(i * 150_000, (i + 1) * 150_000) for i in range(4)]

def timed(label, run):
    t0 = time.perf_counter()
    total = run()
    print(f"{label:<12} {total} primes  {time.perf_counter() - t0:5.2f} s")

if __name__ == "__main__":                      # required: worker processes import this file
    print("GIL enabled:", sys._is_gil_enabled(), "| cores:", os.cpu_count())
    timed("sequential", lambda: sum(count_primes(lo, hi) for lo, hi in CHUNKS))
    with ThreadPoolExecutor(4) as ex:
        timed("4 threads", lambda: sum(ex.map(count_primes, *zip(*CHUNKS))))
    with ProcessPoolExecutor(4) as ex:
        timed("4 processes", lambda: sum(ex.map(count_primes, *zip(*CHUNKS))))
```

its real output on the machine that built this book

```
$ python3 primes.py
GIL enabled: True | cores: 8
sequential   49098 primes   1.16 s
4 threads    49098 primes   1.12 s
4 processes  49098 primes   0.48 s
```

HONEST NOTE

Processes cost more: each starts a fresh interpreter, and arguments and results are pickled between them, so only send small inputs and get back small outputs. Python 3.13+ also ships an optional **free-threaded** build (`python3.14t`, PEP 703/779) with no GIL, where threads do run CPU work in parallel. It's officially supported from 3.14 but not the default, and C extensions must opt in. Locks matter even more there.

asyncio: thousands of waits on one thread

`asyncio` runs many tasks on **one** thread. An `async def` function is a coroutine; at each `await` it pauses and the **event loop** runs another task that's ready. No thread switches you out by surprise: tasks switch only at `await`, so races are much rarer.

```
1  import asyncio, time
2
3  async def fetch(name, delay):
4      await asyncio.sleep(delay)              # pause here; the loop runs other tasks
5      return f"{name} done"
6
7  async def main():
8      t0 = time.perf_counter()
9      results = await asyncio.gather(fetch("a", 0.2), fetch("b", 0.1), fetch("c", 0.15))
10     print(results, round(time.perf_counter() - t0, 1), "s")
11
12 asyncio.run(main())                          # start the event loop, run main, stop
```

```
['a done', 'b done', 'c done'] 0.2 s
```

line 9: run all three concurrently; results in argument order

Structured concurrency: TaskGroup and timeouts

```
1  async def main():
2      try:
3          async with asyncio.timeout(0.12):            # a deadline for the whole block
4              async with asyncio.TaskGroup() as tg:     # every task finishes before we leave
5                  fast = tg.create_task(fetch("fast", 0.05))
6                  slow = tg.create_task(fetch("slow", 1.0))
7      except TimeoutError:
8          print("timed out:", fast.result(), "| slow cancelled:", slow.cancelled())
9
10 asyncio.run(main())
```

```
timed out: fast done | slow cancelled: True
```

line 4: if one task fails, the rest are cancelled

The one rule: never call blocking code inside a coroutine. `time.sleep`, `requests.get` or a slow loop freeze the whole loop, and every other task with it:

time.sleep: blocks the loop

```
import asyncio, time
async def job():
    time.sleep(0.1)
async def main():
    t0 = time.perf_counter()
    await asyncio.gather(job(), job(), job())
    print(round(time.perf_counter() - t0, 1))
asyncio.run(main())
```

```
0.3
```

await asyncio.sleep: yields

```
import asyncio, time
async def job():
    await asyncio.sleep(0.1)
async def main():
    t0 = time.perf_counter()
    await asyncio.gather(job(), job(), job())
    print(round(time.perf_counter() - t0, 1))
asyncio.run(main())
```

```
0.1
```

RULE OF THUMB

Stuck with a blocking library in async code? `await asyncio.to_thread(blocking_fn, args)` runs it in a thread and awaits the result, so the loop keeps going.

An async chat server, end to end

This is where asyncio shines: a server holding many idle connections. Each client gets a coroutine; while it waits for that client's next line, the loop serves everyone else. Two clients connect and one talks:

```
1  import asyncio
2
3  clients = set()
4
5  async def handle(reader, writer):                    # one coroutine per client
6      clients.add(writer)
7      try:
8          while line := await reader.readline():       # b"" when the client leaves
9              for w in clients - {writer}:
10                 w.write(line)
11                 await w.drain()                      # wait if their buffer is full
12     finally:
13         clients.discard(writer)
14         writer.close()
15
16 async def main():
17     server = await asyncio.start_server(handle, "127.0.0.1", 0)
18     port = server.sockets[0].getsockname()[1]
19     async with asyncio.timeout(5):
20         r1, w1 = await asyncio.open_connection("127.0.0.1", port)
21         r2, w2 = await asyncio.open_connection("127.0.0.1", port)
22         await asyncio.sleep(0.05)                    # let the server register both
23         w1.write(b"hi from ana\n")
24         print(await r2.readline())
25         w1.close(); w2.close()
26     server.close()
27
28 asyncio.run(main())
```

```
b'hi from ana\n'
```

line 8: framing by newline (Ch 15)

line 11: backpressure

	threads	asyncio	processes
switches	anywhere (preemptive)	only at `await` (cooperative)	separate programs
shared memory	yes: needs locks	yes, safer: no surprise switches	no: pickle messages
cost each	~MB of stack	~KB	a whole interpreter
CPU-bound speed-up	no (GIL), yes when free-threaded	no	**yes**

works with blocking libraries	**yes**	only via `to_thread`	**yes**

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 16.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 16.1` shows other valid solutions and their trade-offs.

`16.1` fix the bug Make the bank thread-safe (and deadlock-free)
`16.2` build from a spec Run jobs concurrently, at most N at a time
`16.3` build from a spec A concurrent crawler with a work queue
`16.4` build from a spec asyncio: limited, with a timeout each
`16.5` learn an unseen module Debounce with threading.Timer
`16.6` mini-project An asyncio chat server with nicknames

PASSPORT STAMP ask the same question of every new language (Appendix B)

11. Concurrency

Threads share memory (protect it with `Lock`, or pass data through `queue.Queue`). The GIL stops CPU parallelism in threads on the default build; use processes for CPU work. `asyncio`: one thread, cooperative tasks that switch at `await`; `TaskGroup` and `asyncio.timeout` keep them structured.

12. Stdlib and tooling

`threading`, `queue`, `concurrent.futures`, `multiprocessing`, `asyncio`, `contextvars`.

IN OTHER LANGUAGES

Java and C#: real parallel threads, `synchronized`/`lock`, executors and futures. JavaScript: one thread and an event loop, exactly asyncio's model (`async/await` came from C#). Go: goroutines (cheap threads) and channels (queues). Rust: threads with the compiler refusing to build data races, plus async with tokio.

DEEP DIVE (your vault)

Vault notes: 14.1 Concurrency is a lifetime relation · 14.2 Interleaving and race conditions · 14.4 Threads · 14.5 The CPython global interpreter lock · 14.6 Thread communication with queues · 14.7 Processes · 14.9 Asynchronous I/O · 14.10 Structured concurrency with task groups · 14.11 Cancellation and timeouts · 14.12 Backpressure · 14.13 Choosing a concurrency model · 15.16 The GIL and free-threaded internals.